

# Software Development Life Cycle (SDLC)

---

## 1) What is SDLC?

**SDLC** is a step-by-step way to plan, build, test, and ship software. It reduces chaos, helps teams split work, and improves quality.

### Why it exists:

When software became complex (1960s–70s), teams needed a repeatable process to avoid missed requirements, buggy releases, and schedule slips.

---

## 2) A Short History

- **Early days (1960s):** Ad-hoc coding → many failures, delays.
  - **Waterfall (1970s):** A phased, “finish one step before the next” approach became popular for predictability.
  - **Iterative/Spiral (1980s–90s):** Add risk-driven loops and partial builds.
  - **Agile (2001 → ):** Short cycles (sprints), continuous feedback, frequent releases.
  - **CI/CD (2010s → ):** Automate build, test, and deploy → ship safely every day.
- 

## 3) Waterfall Model (the “classic” SDLC)

### Phases, in order:

1. Requirements
2. Design
3. Implementation (coding)
4. Testing
5. Deployment
6. Maintenance

**Strengths:** Clear plan, heavy documentation, easy to manage for fixed scope.

**Limitations:** Late feedback (you test at the end), expensive to change requirements, slow to deliver value.

Use for: **well-defined, stable** projects (e.g., compliance or hardware-tied systems).

---

---

## 4) Agile Model (why the industry moved)

**Core idea:** Work in **short iterations** (1-2 weeks). Build small pieces, get feedback, adjust quickly.

**Solves:**

- Requirements change? Iterate next sprint.
- Early demos reduce wrong assumptions.
- Continuous testing keeps quality high.

**Common Agile practices:**

- **Scrum roles:** Product Owner (what to build), Scrum Master (process), Dev Team (build & test).
  - **Ceremonies:** Sprint Planning, Daily Stand-ups, Sprint Review (demo), Retrospective.
  - **Artifacts:** Product Backlog, Sprint Backlog, Increment (shippable product each sprint).
- 

## 5) CI/CD (Continuous Integration / Delivery / Deployment)

- **CI:** Every code change is merged to main (or trunk) frequently. A bot **builds & tests automatically**.
- **CD (Delivery):** After CI, a **release package** is always ready to deploy with one click.
- **CD (Deployment):** Push to production **automatically** after passing checks (some teams still keep a manual approval).

**Benefits:** Fast, reliable releases; small changes → easier debugging; fewer “big bang” failures.

**Typical pipeline steps:** Lint → Unit tests → Build → Integration/E2E tests → Security/Quality checks → Package → Deploy → Smoke tests → Monitor.

---

## 6) The SDLC Phases (what you actually do)

### 6.1 Requirements

**Goal:** Agree on *what* to build.

- **Elicit:** talk to users, mentors, sample customers.
- **Functional requirements:** features (e.g., “Users can create surveys with MCQ and rating questions.”)

- **Non-functional:** performance, security, privacy, scalability, usability, accessibility, cost.
- **Constraints:** time, tech choices, regulations, platform (Android/iOS/Web).

**Deliverables:** Problem statement, user stories, acceptance criteria, scope (in/out), basic timeline.

---

## 6.2 Analysis

**Goal:** Understand the problem deeply and model it.

- **Use cases** and **user flows**
- **Data model** candidates (entities, relationships)
- **Risks & assumptions** (e.g., "Assume internet connectivity; risk: high traffic at peak hours")

**Deliverables:** Use-case list, context diagram, early ERD (entity-relationship diagram).

---

## 6.3 Design

Split design into layers:

- **Software/System Architecture:** frontend, backend, database, auth, messaging, 3rd-party services. Choose patterns (e.g., REST APIs, MVC).
- **Component/Module Design:** how pieces talk (APIs, interfaces), error handling, logging.
- **Object-Oriented Design (OOD):** key classes, methods, relationships (UML class diagrams).

**Deliverables:** Architecture diagram, sequence diagrams, ERD, API contracts (OpenAPI/Swagger), UI wireframes.

---

## 6.4 Implementation (Coding)

**Practices:**

- Coding standards & style guides
- Version control (Git): **trunk-based** (team commits to `main` daily) or **feature-branch** + PR reviews
- Small, frequent commits; meaningful messages
- Dependency management; environment variables for secrets
- Feature flags for safe rollouts

**Deliverables:** Working code, PRs, code reviews, commit history.

---

## 6.5 Testing & Quality

**Types of tests (keep them fast & automated where possible):**

| Level                   | What it checks        | Example                                          |
|-------------------------|-----------------------|--------------------------------------------------|
| Unit                    | Single function/class | Validate email, compute score                    |
| Integration             | Modules together      | API ↔ DB interactions                            |
| End-to-End (E2E)        | Full user flow        | Create survey → submit response → view analytics |
| Performance             | Speed/throughput      | Load test submit endpoint                        |
| Security                | Vulnerabilities       | Input validation, OWASP checks                   |
| Usability/Accessibility | UX & a11y             | Keyboard nav, ARIA labels, color contrast        |
| Regression              | Nothing broke         | Re-run test suite after changes                  |
| Smoke                   | Quick sanity          | App boots, main page loads                       |

**Deliverables:** Test plan, automated test suite, coverage reports, bug list, fixes.

---

## 6.6 Deployment

- **Environments:** Dev → Test/Staging → Production
- **Release methods:** Blue-green, canary, or rolling updates
- **Rollback plan:** Always have a quick revert strategy
- **Observability:** Logs, metrics, traces, alerts, uptime checks

**Deliverables:** Release notes, deployment scripts/configs, runbooks.

---

## 6.7 Maintenance & Iteration

- Monitor real usage, fix bugs, optimize
  - Prioritize new features based on feedback
  - Repeat mini-cycles of plan → build → test → ship
- 

# 7) How to Run a College Project Team

**Recommend:** Light Scrum + CI

- **Sprints:** 1 week (Mon–Sun)
- **Board:** To-do / In-progress / In-review / Done (GitHub Projects/Trello)
- **Branching:** feature branches + PR reviews OR trunk-based + short-lived branches

- **Definition of Done (DoD):** Code reviewed, tests passing, docs updated, demoable
- **Weekly rhythm:**
  - Mon: Plan sprint (pick user stories)
  - Daily: 10-min stand-up (What I did / Will do / Blockers)
  - Fri: Demo + Retro (what to improve)
- **Roles (lightweight):**
  - Lead/PO: scope & priorities
  - Devs: implement & test
  - QA owner: test plans & automation
  - DevOps owner: pipeline, deployments (can be shared)

### **Suggested semester timeline (~10–12 weeks):**

- Weeks 1–2: Requirements + analysis + wireframes
  - Weeks 3–4: Architecture + DB/API design + CI pipeline
  - Weeks 5–8: Implementation in slices (vertical features) + tests
  - Weeks 9–10: Hardening: performance, security, a11y, bug fixes
  - Weeks 11–12: Final report, demo video, user guide, deployment polish
- 

## **8) Worked Example: Survey App**

**Goal:** Build a survey platform with:

- **Frontend:** Mobile-friendly web app (and optional mobile app)
- **Backend:** REST API
- **Database:** Relational (e.g., Postgres/MySQL)

### **8.1 Requirements (sample)**

#### **Functional:**

- Create surveys (title, description, start/end dates)
- Question types: MCQ (single/multiple), rating (1–5), short text
- Share link/QR; allow anonymous or authenticated responses
- View analytics: total responses, per-question charts, export CSV

#### **Non-Functional:**

- Handle 100 concurrent users
- P95 API latency < 300ms
- Secure (no PII leak), input validation, rate limiting
- Responsive design (mobile first), accessible (WCAG basics)

### Acceptance criteria example (user story):

"As a survey owner, I can add MCQ questions so that respondents can choose one or many options."

- Given I am editing a survey, when I add an MCQ question with options, then it appears in the preview and can be answered.
- 

## 8.2 Analysis

**Key entities:** User, Survey, Question, Option, Response, Answer, Invite/Link

### High-level use cases:

- Owner creates/edits a survey
  - Respondent opens link and submits responses
  - Owner views analytics dashboard
- 

## 8.3 Design

### Architecture (typical):

- **Frontend:** React/Vue (Responsive UI)
- **Backend:** Node/Express, Python/FastAPI, or Java/Spring Boot
- **DB:** Postgres
- **Auth:** Email/OTP or Google OAuth (for owners)
- **Storage:** Object storage for exports (optional)
- **CI/CD:** GitHub Actions → Deploy to Render/Heroku/Vercel/Fly.io

### API sketch (examples):

- `POST /api/v1/surveys` – create survey
- `POST /api/v1/surveys/:id/questions` – add question
- `GET /api/v1/surveys/:id/public` – public schema for respondents
- `POST /api/v1/surveys/:id/submit` – submit answers
- `GET /api/v1/surveys/:id/analytics` – aggregated stats

### DB schema (minimal):

```
User(id, name, email, created_at)
Survey(id, owner_id -> User, title, description, starts_at, ends_at,
is_public, created_at)
Question(id, survey_id -> Survey, type, text, required, order_index)
Option(id, question_id -> Question, text, order_index) --
```

```
for MCQ
Response(id, survey_id -> Survey, respondent_id nullable, created_at) --
anonymous allowed
Answer(id, response_id -> Response, question_id -> Question,
      selected_option_id nullable -> Option, free_text nullable, rating
nullable)
```

### OOD hint:

Classes like `Survey`, `Question`, `Response`, `Answer`, with methods `addQuestion()`, `validate()`, `computeAnalytics()`.

### UI wireframes:

- Owner dashboard: My Surveys / Create New
  - Builder: Add/Order questions, Preview
  - Public form: Minimal, keyboard-friendly, mobile-first
  - Analytics: charts per question, CSV export
- 

## 8.4 Implementation

### Tech choices (example):

- Frontend: React + Router + state mgmt (light)
- Backend: FastAPI/Express + JWT (owner), rate limiting (respondents)
- DB migrations: Prisma/TypeORM/Flyway
- Validation: shared schema (e.g., JSON Schema / Zod)
- Feature flags: enable/disable rating type

### Git workflow:

- Branch per feature: `feat/surveys-create`, `feat/submit-endpoint`
  - PR template: Summary, screenshots, tests, checklist
  - Code review: At least 1 approval, CI green
- 

## 8.5 Testing

### Unit tests:

- Validate question payloads, answer parsing, analytics math

### Integration tests:

- API ↔ DB: create survey → add questions → submit response → query analytics

### E2E tests (Playwright/Cypress):

- Owner creates a survey and sees it live
- Respondent completes a survey on mobile viewport

### Performance tests (k6):

- Load test `/submit` for 100 RPS, check error rate & latency

### Security checks:

- Input sanitization (XSS), auth on owner endpoints, rate limit on submit, CSRF for owner UI

### Accessibility:

- Tab order, labels, color contrast (check with Lighthouse/axe)
- 

## 8.6 Deployment

### Environments:

- Dev (auto deploy on `main`)
- Staging (tagged release)
- Prod (manual approval or canary)

### Strategy:

- **Blue-green:** switch traffic after smoke tests
- **Rollback:** keep previous version; one-click revert

### Observability:

- Logs (request ids), metrics (throughput, latency), alerts (error spikes)
- 

## 8.7 Iterate

- Collect feedback from real users (classmates/invitees)
  - Prioritize: bug fixes → UX issues → new question types
  - Plan next sprint; keep shipping small slices
- 

# 9) What Your Project Report Should Contain

Use this as a ready checklist for your submission.



1. **Title Page:** Project name, team members (roles), guide, department, semester, date
2. **Abstract (≤200 words):** Problem, approach, key results
3. **Introduction:** Context, motivation, goals, scope
4. **Literature/Related Work:** Similar tools, what's missing
5. **Requirements:** Functional & non-functional; user stories; acceptance criteria
6. **Analysis:** Use cases, user flows, assumptions, risks
7. **Design:**
  - Architecture diagram
  - ERD (database)
  - API contracts (endpoints, request/response)
  - UML (class/sequence)
  - UI wireframes
8. **Implementation:** Tech stack, modules, key algorithms, code structure, environment setup
9. **Testing:** Test plan, types of tests, tools used, coverage, key results, bug list & fixes
10. **Performance & Security:** Load results, optimizations, security measures
11. **Deployment:** Environments, CI/CD pipeline, release steps, rollback plan
12. **Results & Discussion:** Screenshots, analytics, what worked, what didn't
13. **Conclusion & Future Work:** Learnings, next steps (e.g., new question types, analytics)
14. **Team Contributions:** Who did what (per sprint or module)
15. **Timeline:** Gantt/burndown chart, milestones
16. **User Guide:** How to run locally, how to deploy, how to use the app
17. **References:** Tutorials, docs, libraries, papers
18. **Appendices:** Full test cases, sample data, additional diagrams, major code listings

**Tip:** Include a **demo video link** (2–5 minutes): problem → features → short live demo.

---

## 10) Quick Checklists

### 10.1 Sprint Planning

- ☐ Pick user stories with acceptance criteria
- ☐ Break into tasks (≤ 1 day each)
- ☐ Define DoD (tests, docs, review)
- ☐ Update board (assignees, due dates)

### 10.2 PR Checklist

- ☐ Clear title & summary
- ☐ Screenshots (UI changes)
- ☐ Unit/Integration tests added

- ☐ Lint & CI passing
- ☐ Backwards compatible (migrations, APIs)
- ☐ Docs updated

## 10.3 Release Readiness

- ☐ Version tagged & changelog written
  - ☐ Smoke tests green in staging
  - ☐ Monitoring dashboards ready
  - ☐ Rollback plan documented
- 

## 11) Common Pitfalls (and fixes)

- **Huge scope:** Start with MVP; add features later.
  - **Late testing:** Write tests from the first sprint; automate in CI.
  - **Weak documentation:** Update README and API docs every sprint.
  - **No demo until the end:** Demo every week to get feedback.
  - **Single "hero" coder:** Share knowledge; pair program; code reviews.
- 

## 12) Minimal MVP for the Survey App

- Create survey with **MCQ + rating + short text**
- Public link to answer
- See **count of responses** and **per-question distribution**
- CSV export

Ship this first, then add:

- Auth, schedules, themes, more analytics, mobile app, etc.
- 

## 13) Suggested Tools (student-friendly)

- **Planning/Docs:** GitHub Projects/Issues, Notion/Google Docs, Excalidraw/Draw.io for diagrams
- **Version Control:** GitHub/GitLab
- **CI/CD:** GitHub Actions (build, test, deploy)
- **Backend:** FastAPI / Express / Spring Boot
- **Frontend:** React / Vue; Tailwind or simple CSS
- **DB:** Postgres (with Prisma/TypeORM)

- **Tests:** PyTest/Jest/Vitest, Playwright/Cypress, k6(perf)
  - **Hosting (free/low-cost):** Vercel/Netlify (frontend), Render/Fly.io/Heroku (API), Neon/Supabase (Postgres)
- 

## 14) One-Page Summary (for your wall)

- **Plan small → Build small → Test always → Ship often.**
  - Favor **Agile + CI** over big waterfall plans (unless scope is fixed & tiny).
  - Keep **docs, tests, and demo** updated weekly.
  - **MVP first**, then iterate with feedback.
  - End with a **clear, complete project report** and a **short demo video**.
- 

## Appendix A – Sample Acceptance Criteria (Survey Creation)

Given I am logged in as the owner

When I create a survey with a title and one MCQ question

Then the survey saves successfully

And I can preview the survey with that MCQ visible

And anonymous respondents can submit answers via the public link

## Appendix B – Sample CI Workflow (high level)

```
on: pull_request
```

```
jobs:
```

```
  build-test:
```

```
    steps:
```

- checkout
- setup runtime (node/python/java)
- install deps
- lint
- unit & integration tests
- build artifacts
- (optional) run E2E on ephemeral env